

LLM Training with LoRA — Continued Pre-training and SFT

Part 1 – Submission due 24.5.2026

In the following two assignments you will practice training a language model in two stages — continued pre-training and instruction fine-tuning. In both cases you will need to evaluate the models and write conclusions.

In this part, we will focus on continued pre-training of a small language model. Keep in mind that Part 2 will depend on Part 1: we will take our pre-trained model and fine-tune it.

General Instructions:

- You are encouraged to use every tool at your disposal, including AI coding assistants (Claude, Copilot, ChatGPT - please add a paragraph to your report describing how you used the AI assistant in this task.
- We have allocated a special queue in the department lab for this course. To use it, please use the following settings:

```
qos course, --part rtx2080, --gpu 1, --account cours_ibm26
```

. This GPU should be enough for the tasks, but you are free to use larger GPUs or Colab.
- The submission must include both a report and the code you used. You are free to use notebooks or Python scripts. The report (PDF file) must include:
 - Everything explicitly requested in the assignment.
 - Which GPUs did you use.
 - All assumptions you took.
 - [Important] What did not work: log your mistakes. This is a very important step.

Task:

The task is to adapt a pre-trained LM to a given corpus using LoRA, and produce evidence that adaptation happened and improved in-domain performance.

1. **Environment:**

We are not enforcing any special packages, but our recommendation would be torch, transformers, peft, datasets, bitsandbytes, accelerate and trl.

- When installing keep in mind the version of the CUDA in the cluster (probably > 13) and will require newer versions of the packages – shouldn't matter if you are creating new environment.
- While we haven't taught this in class, we encourage you to read about Unsloth (<https://unsloth.ai/>) which is a python package that helps saving memory in training. It can allow you to use bigger models with the same settings. (you can complete the task without unsloth)

2. **Model choice:**

First, you would need to pick an open-source model. You can pick whatever model you wish to from huggingface.

Before selecting a model for fine-tuning, add a section to the report containing a table in which each row corresponds to a different model size: 1B, 3B, 8B, and 13B. For each model size, calculate and report:

- Weights-only footprint. How much GPU memory the model occupies as-is - just the weights, no training state.
- Training footprint, full precision vs. 4-bit. How much GPU memory the model requires during training, reported for both full-precision fine-tuning and 4-bit (QLoRA-style) fine-tuning.
- Auxiliary memory components. Which other components of the training step consume GPU memory (e.g., gradients, optimizer state, activations, communication buffers), and how much each contributes.
- Effect of max_length on memory. How memory consumption scales with sequence length, computed for max_length values of 256, 512, 1024, and 2048.

You do not need to actually run these models. Estimate memory using formulas and LLMs, we just want you to get the sense of sizes.

Examples for models you can pick:

- IBM-granite: <https://huggingface.co/ibm-granite>
- Google-Gemma: <https://huggingface.co/google>
- Qwen: <https://huggingface.co/Qwen>
- There are more, you can search – You are free to go smaller or larger if you fit.

3. Data:

The corpus: Sherlock Holmes canon

Your corpus is the nine canonical Sherlock Holmes books by Arthur Conan Doyle, all in the public domain on Project Gutenberg:

#	Title	Gutenberg ID
1	A Study in Scarlet	244
2	The Sign of the Four	2097
3	The Hound of the Baskervilles	2852
4	The Valley of Fear	3289
5	The Adventures of Sherlock Holmes	1661
6	The Memoirs of Sherlock Holmes	834
7	The Return of Sherlock Holmes	108
8	His Last Bow	2350
9	The Case-Book of Sherlock Holmes	69700

For the task you will need to download the data from <https://www.gutenberg.org/cache/epub/<ID>/pg<ID>.txt> (for example, A Study in Scarlet is at <https://www.gutenberg.org/cache/epub/244/pg244.txt>)

You should reserve ~5% of the corpus as held-out validation for Task 1 - ideally a whole chapter or story, not random sentences, so perplexity is measured on coherent continuations.

Once we got the data we would like to make a small preprocessing stage. You must do the following:

- Count the number of tokens in the corpus

- Chunking strategy – explain how you decided to split the corpus into training samples.
- Histogram of input length – Does the input length distribution fit within the model's maximum context length?
- Are there any special characters that needs to be cleaned before training?
- You are free to add more preprocessing stages as you wish.
- You are free to add more training data as you wish.

4. **Training:**

In this stage you will run the actual training. The training framework is TRL (<https://huggingface.co/docs/trl/index>), following the script shown in class.

- Configure LoRA with PEFT; rank, alpha, target modules.
- Define the training arguments (epochs, LR, optimizer, etc.)

5. **Experiments:**

This is the key part of the assignment; in this stage you will run experiments on the task and try to understand which training configuration works best for this corpus and model.

You must pick at least three of these ablations. For each ablation, state your hypothesis before running it, and reconcile the result with your hypothesis in the report – a surprising result explained honestly is worth more than a confirmed one.

- LoRA rank changes
- LoRA target modules
- Learning rate sensitivity
- QLoRA vs full LoRA
- Warmup and Learning rate scheduler, do they help? How much for each?
- Try using CPU-offloading techniques (QLoRA, deepspeed)
- Epoch changes, do they help?

You can add experiments as you wish!

6. **Evaluation:**

In pre-training the main metric will be perplexity.

(<https://huggingface.co/docs/transformers/perplexity>):

- You must evaluate both the model before training and after training
- It is important to follow and monitor the learning process using TensorBoard, Weights & Biases, trackio or any other framework you wish to use.
- [Not mandatory] adding another trained model- see how they compete.